

QA Strategy & Test Approach

Author: Kevin Velasco

Document Type: QA Strategy / Test Approach

Purpose: Portfolio Demonstration of QA Planning and Quality Practices

1. Document Overview

This document defines a practical QA strategy and test approach for a software development environment where quality activities need to be structured, prioritized, and communicated effectively.

The approach is designed around a key principle:

Quality is not only about finding defects. It is about making informed decisions, managing risk, and making quality visible throughout the software development lifecycle.

As a QA Engineer, my approach is to establish clear testing priorities, collaborate with developers and product stakeholders, document decisions, and ensure that critical user journeys receive appropriate coverage.

2. QA Strategy

A QA strategy should provide a clear framework for how testing will be planned, executed, communicated, and improved.

The strategy focuses on:

- Risk-based testing
- Collaborative test planning
- Developer collaboration
- Functional and exploratory testing
- Regression testing
- Defect management
- Test environments
- CI/CD integration
- QA documentation and ownership

The goal is to balance **test coverage, business risk, available resources, and release timelines** rather than attempting to test every possible scenario.

3. Risk-Based Testing

Objective

Testing resources should be focused on areas where a defect could have the greatest impact.

Because it is not practical to test everything with equal depth, risk-based testing provides a defensible basis for deciding which functionality requires the most attention.

Approach

Risk is evaluated based on factors such as:

- Business impact
- User impact
- Likelihood of failure
- Complexity of the functionality
- Frequency of use
- Dependencies with other systems

Testing Priority

High-risk functionality receives deeper testing and stronger regression coverage, while lower-risk areas may receive lighter or exploratory coverage.

For example, critical user journeys such as **login, checkout, and payment** should receive higher testing priority than low-impact UI changes.

Risk therefore becomes a shared language between QA, Product, and Engineering when testing scope and release trade-offs need to be discussed.

4. Collaborative Test Planning

Testing should begin before implementation is complete.

QA should collaborate with developers and product stakeholders during planning and development to identify:

- Expected behavior
- Acceptance criteria
- Potential edge cases

- Business risks
- Integration points
- Testability concerns
- Potential failure scenarios

Early collaboration helps identify quality issues before they become expensive to fix later in the development cycle.

The objective is to make testing part of the development process rather than an activity performed only after implementation.

5. Developer Collaboration and Pair Testing

Working directly with developers can improve both efficiency and product understanding.

Pair testing may involve:

- Reviewing functionality together
- Reproducing defects
- Exploring edge cases
- Validating fixes
- Investigating unexpected behavior
- Discussing technical limitations and dependencies

This approach allows QA and developers to share knowledge and resolve issues faster while maintaining a common understanding of expected quality.

6. Quality Thinking Framework

QA should go beyond simply asking:

"Does it work?"

Testing should also ask:

Expected Behavior

Should it work this way?

Does the implementation satisfy the intended business requirement?

Unexpected User Behavior

What happens when users do something we did not anticipate?

This includes invalid inputs, unexpected sequences, boundary conditions, and unusual user behavior.

Business Value

Is the solution actually solving the problem it was intended to solve?

Testing should consider not only technical correctness but also whether the delivered functionality meets the intended user and business needs.

7. QA Workflow

Quality activities should be integrated throughout the software development lifecycle.

Discovery / Requirements

QA activities include:

- Understanding business requirements
- Reviewing acceptance criteria
- Identifying risks
- Identifying test scenarios
- Raising questions and ambiguities early

Before Implementation

QA collaborates with Product and Engineering to establish:

- Test scope
- Expected behavior
- Acceptance criteria
- Potential edge cases
- High-risk areas

During Development

QA activities include:

- Reviewing implemented functionality
- Performing exploratory testing
- Collaborating with developers
- Reporting and reproducing defects
- Validating fixes

Pre-Release

QA focuses on:

- Regression testing
- Critical user journeys
- High-risk functionality
- Defect verification
- Release readiness

Post-Release

QA activities may include:

- Monitoring reported issues
- Validating production fixes
- Reviewing escaped defects
- Updating regression coverage
- Improving future test strategies

This approach ensures that quality is considered **before, during, and after implementation**, rather than being treated as a final gate before release.

8. Test Approach

The test approach combines multiple testing techniques depending on risk and functionality.

Functional Testing

Validate that the application behaves according to defined requirements and acceptance criteria.

Exploratory Testing

Explore the application beyond predefined test cases to identify unexpected behavior and edge cases.

Regression Testing

Verify that changes and fixes have not negatively affected existing functionality.

Edge-Case Testing

Focus on unusual, invalid, boundary, and unexpected user scenarios.

The goal is to combine structured testing with investigative testing rather than relying exclusively on predefined test cases.

9. Defect Management

Defects should be documented clearly and prioritized according to their impact.

Severity

Severity describes the **impact of the defect on the system or users**.

Example levels:

- **Critical** — Prevents a critical business process or causes severe system impact.
- **High** — Major functionality is unavailable or significantly affected.
- **Medium** — Functionality is affected but a workaround may exist.
- **Low** — Minor functional or UI issue with limited impact.

Priority

Priority describes **how soon the defect should be addressed**.

A low-severity issue may still have high priority if it affects an important release or customer-facing workflow.

Severity and priority should therefore be considered separately when deciding how defects are handled.

10. SLAs and Communication Rules

Clear communication rules help the team make consistent decisions.

The QA process should establish expectations around:

- When defects should be addressed
- Who determines release readiness
- How defects are prioritized
- Severity definitions
- Priority definitions
- Escalation of critical issues
- Communication of release risks

The purpose is to ensure that QA findings are understood and actionable rather than simply being added to a defect tracker.

11. Core Regression Pack

A core regression suite should focus on the application's most important user journeys.

Examples include:

- Login
- Checkout
- Payment
- Other business-critical workflows

Critical regression scenarios should be strong candidates for automation where appropriate.

Automation provides repeatable coverage for stable workflows, while manual testing remains valuable for:

- Exploratory testing
- New functionality
- Edge cases
- Usability observations
- Scenarios that are not cost-effective to automate

The objective is not to automate everything, but to automate the **right tests**.

12. Test Environments and CI/CD

Testing should be performed in environments appropriate to the development lifecycle.

Typical environments may include:

- Development
- Staging
- Production

Where CI/CD is available, automated tests should be integrated into the delivery pipeline so that defects can be detected earlier.

The goal of CI/CD testing is not simply to act as a gatekeeper. It should provide **fast feedback about software quality throughout the development process**.

13. QA Documentation

Documentation provides continuity and makes QA practices understandable to the rest of the organization.

Important QA documentation may include:

- Test strategies
- Test plans
- Test cases
- Risk assessments
- Regression suites
- Defect management guidelines
- Severity and priority definitions
- Test results
- Release-readiness information
- Ownership and responsibilities

Good documentation ensures that QA knowledge does not remain dependent on one person.

It also allows future team members to understand the testing approach without having to rebuild the process from the beginning.

14. QA Ownership

When working as the primary or only QA resource, discipline and prioritization become especially important.

QA ownership includes:

- Defining the test approach
- Identifying and communicating risks
- Making testing trade-offs explicit
- Maintaining regression coverage
- Communicating quality status
- Documenting processes
- Collaborating with Engineering and Product
- Continuously improving the testing strategy

The responsibility is not simply to execute test cases.

It is to help the organization **understand product quality and make informed release decisions.**